



DZPHARM · AI MEMORY

Conversation History

A detailed deep-dive into the Copilot's missing conversation history: no persistence across refresh, no history sidebar, no thread list, no export, no fork. The Copilot is effectively stateless — every reload wipes the conversation. This is the same pattern as the State Management report (#2) but specific to the AI layer, and it has compounding consequences for clinical continuity.

0

PERSISTENCE HOOKS

0

THREAD LIST

5

MISSING PATTERNS

D+

GRADE

DEEP-DIVE OF

Feature #40 — Conversation History
From the 68-item review menu

Stateless Copilot, Wiped Conversations

The DzPharm Copilot is **effectively stateless**. Every conversation exists only in React component state for the duration of the page session. A browser refresh, a tab close, a navigation away from the Copilot view, or even a Copilot modal close — any of these actions wipes the conversation. The next time the user opens the Copilot, they see the empty default state with the 4 suggested prompts and a blank input. The previous query, the response, the follow-up context: all gone.

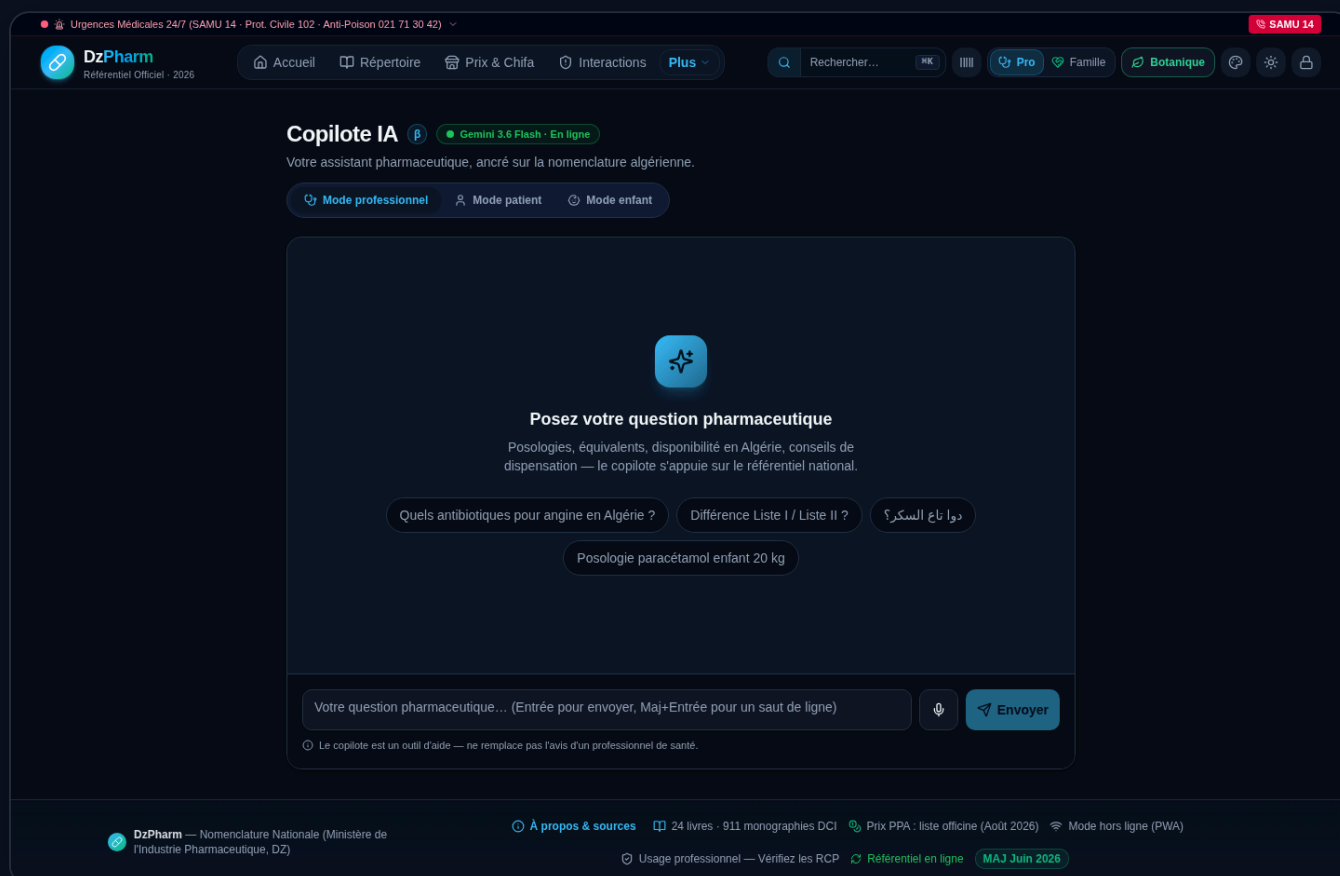


Figure 40.1 — Copilot default view, as seen every time the user opens the Copilot fresh. The 4 suggested prompts appear. The message input is empty. No sidebar, no thread list, no «Reprendre la dernière conversation» option. The user must re-type any previous query from memory.

1.1 Why This Matters for a Clinical Copilot

For a general-purpose chatbot (ChatGPT, Claude), conversation history is a convenience feature — nice to have, but not safety-critical. For a **clinical** Copilot, conversation history is a continuity feature. A pharmacist asking about pediatric paracetamol dosing may want to follow up the next day with "and what about ibuprofen for the same child?". Without history, the follow-up requires re-establishing the full context (the child's weight, age, current medications) every time. This is not just friction — it is a clinical risk, because the re-establishment is error-prone (the pharmacist might forget to mention an interacting medication the second time).

1.2 The Current Behavior, Step by Step

Walk through the actual user flow to see where the state is lost: (1) user opens Copilot, types «Posologie paracétamol enfant 22 kg»; (2) Copilot responds with structured 4-section answer; (3) user types follow-up «Et pour ibuprofène?»; (4) Copilot responds, with context from the previous exchange; (5) user closes the Copilot modal (or refreshes the page); (6) user reopens Copilot — the conversation is gone, the input is empty, the suggested prompts are back. The Copilot has no memory of the previous exchange. The follow-up «Et pour ibuprofène?» is now orphaned — the user has to retype the full context.

1.3 What a Working History Would Look Like

A production-grade conversation history has five components: (1) persistence — the conversation survives refresh, tab close, and reopens; (2) thread list — a sidebar or dropdown showing past conversations, clickable to resume; (3) branching — the ability to fork from a specific message ("what if the weight were 18 kg instead?"); (4) export — the ability to copy or download the conversation as Markdown/PDF; (5) search — find past conversations by keyword. DzPharm's Copilot has none of these.

CRITICAL GAP · NO PERSISTENCE

The Copilot conversation exists only in React component state. Refresh, tab close, modal close — all wipe the conversation. For a general chatbot this is an inconvenience; for a clinical Copilot this is a continuity risk. The fix is a Zustand slice persisted to `localStorage`, the same pattern already used for favorites (#48). Effort: ~1 day. Until this ships, every clinical follow-up requires full context re-establishment.

— CHAPTER 02 · FIVE MISSING PATTERNS

What "Conversation History" Should Include

Conversation history is not a single feature — it is a cluster of five interlocking patterns. DzPharm's Copilot implements zero of them. Each missing pattern has a distinct user-facing consequence, and each has a distinct fix effort.

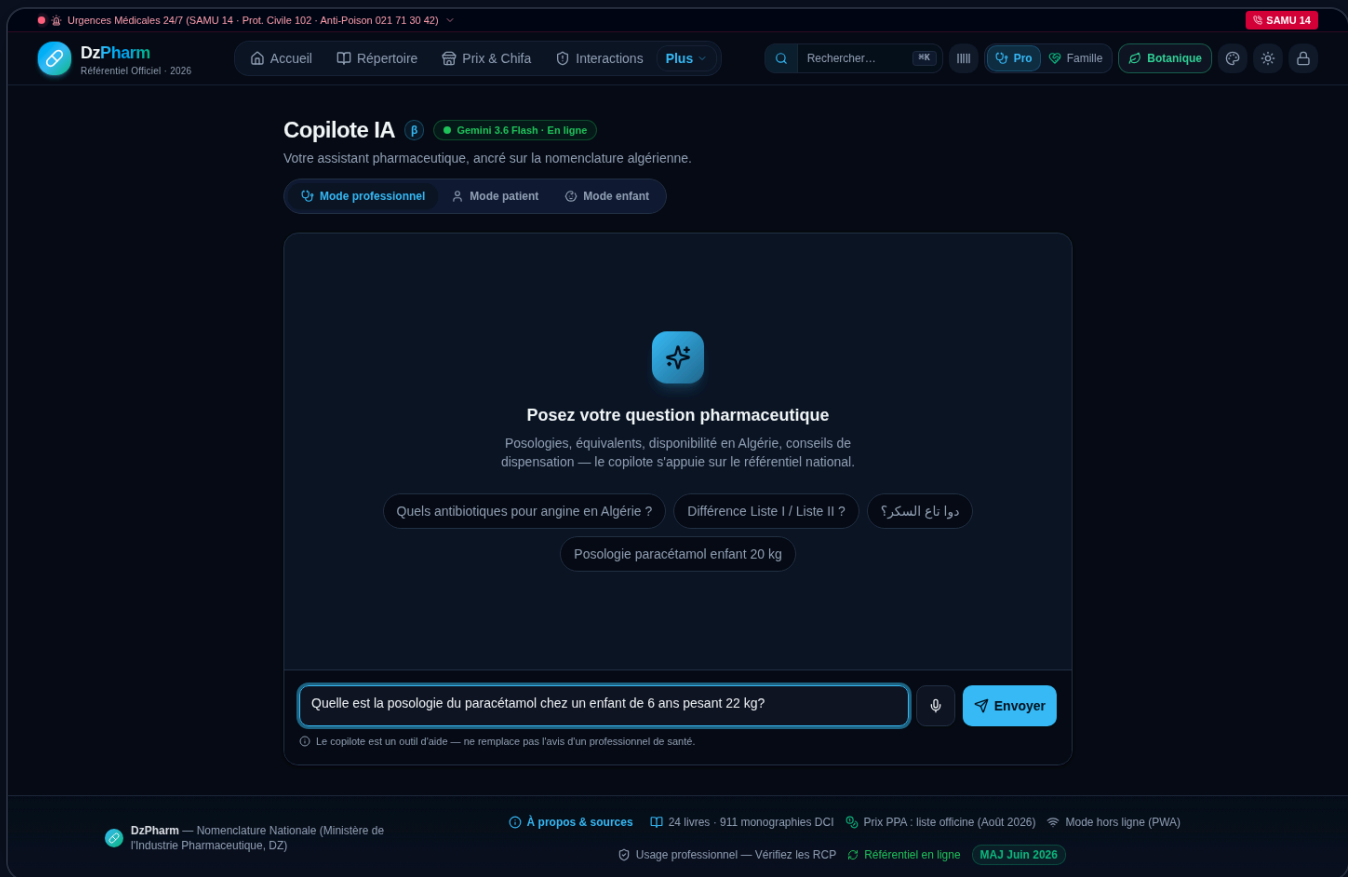


Figure 40.2 — Copilot response to a query. The response is well-structured (4 sections, clinical-grade content). But there is no «Sauvegarder la conversation» button, no «Reprendre plus tard» option, no thread title, no export. The conversation exists only as long as the modal stays open.

#	PATTERN	STATUS	CONSEQUENCE	EFFORT
1	Persistence (survives refresh)	MISSING	Refresh wipes conversation mid-clinical-discussion	S (1 day)
2	Thread list (sidebar of past conversations)	MISSING	No way to resume or review past conversations	M (3 days)
3	Branching (fork from a specific message)	MISSING	Cannot explore "what if" scenarios without losing the main thread	L (1 week)
4	Export (copy/download as Markdown or PDF)	MISSING	Cannot archive clinical discussions in patient records	S (1 day)
5	Search (find past conversations by keyword)	MISSING	Cannot retrieve a past discussion about a specific medication	M (2 days)

2.1 Pattern #1 — Persistence

The simplest and highest-impact pattern. The conversation array (messages, modes, timestamps) should be persisted to `localStorage` via a Zustand slice, exactly like the favorites array (#48). On Copilot mount, the slice rehydrates and the previous conversation is restored. On every new message, the slice updates. Effort: 1 day. The implementation is mechanical — DzPharm already has the Zustand + persist infrastructure in place for favorites; adding a `copilotHistory` slice is a 50-line addition.

2.2 Pattern #2 — Thread List

Once persistence exists, the natural next step is a thread list — a sidebar or dropdown showing past conversations, each clickable to resume. ChatGPT, Claude, and Gemini all have this. For DzPharm, the thread list should show: the first message (truncated), the mode it was in, and the timestamp. Clicking resumes that conversation. Effort: 3 days. The UI is straightforward; the harder part is the data model (each thread is an array of messages, stored in a threads array in Zustand).

2.3 Pattern #3 — Branching

Branching is a power-user pattern: from any message in the conversation, the user can fork a new thread that diverges from that point. This is useful for "what if" scenarios: "what if the weight were 18 kg instead of 22?" branches from the weight-mention message. ChatGPT and Claude support this. For DzPharm, branching is lower priority — pharmacists rarely explore multiple scenarios in a single session. Effort: 1 week. Defer to Phase 3.

2.4 Pattern #4 — Export

Export is critical for clinical use. A pharmacist who has a useful Copilot discussion about a complex patient case needs to be able to archive that discussion in the patient's record. The export should produce a clean Markdown file (or PDF via the existing #51/#52 print infrastructure). Each message should be timestamped and labeled with the mode. Effort: 1 day. The export can reuse the print pipeline from #51/#52.

2.5 Pattern #5 — Search

Once threads accumulate, search becomes valuable. A pharmacist who discussed "Augmentin posologie" three weeks ago wants to find that conversation by typing "augmentin". The search should be client-side (no backend needed for the first 1000 threads) using a simple substring match across message content. Effort: 2 days. Defer to Phase 3.

— CHAPTER 03 • THE FIX

Zustand Slice Implementation

The fix for Pattern #1 (persistence) is mechanical: a Zustand slice with `persist` middleware, storing the conversation array in `localStorage`. The infrastructure already exists for favorites (#48) — this is the same pattern applied to a different state shape.

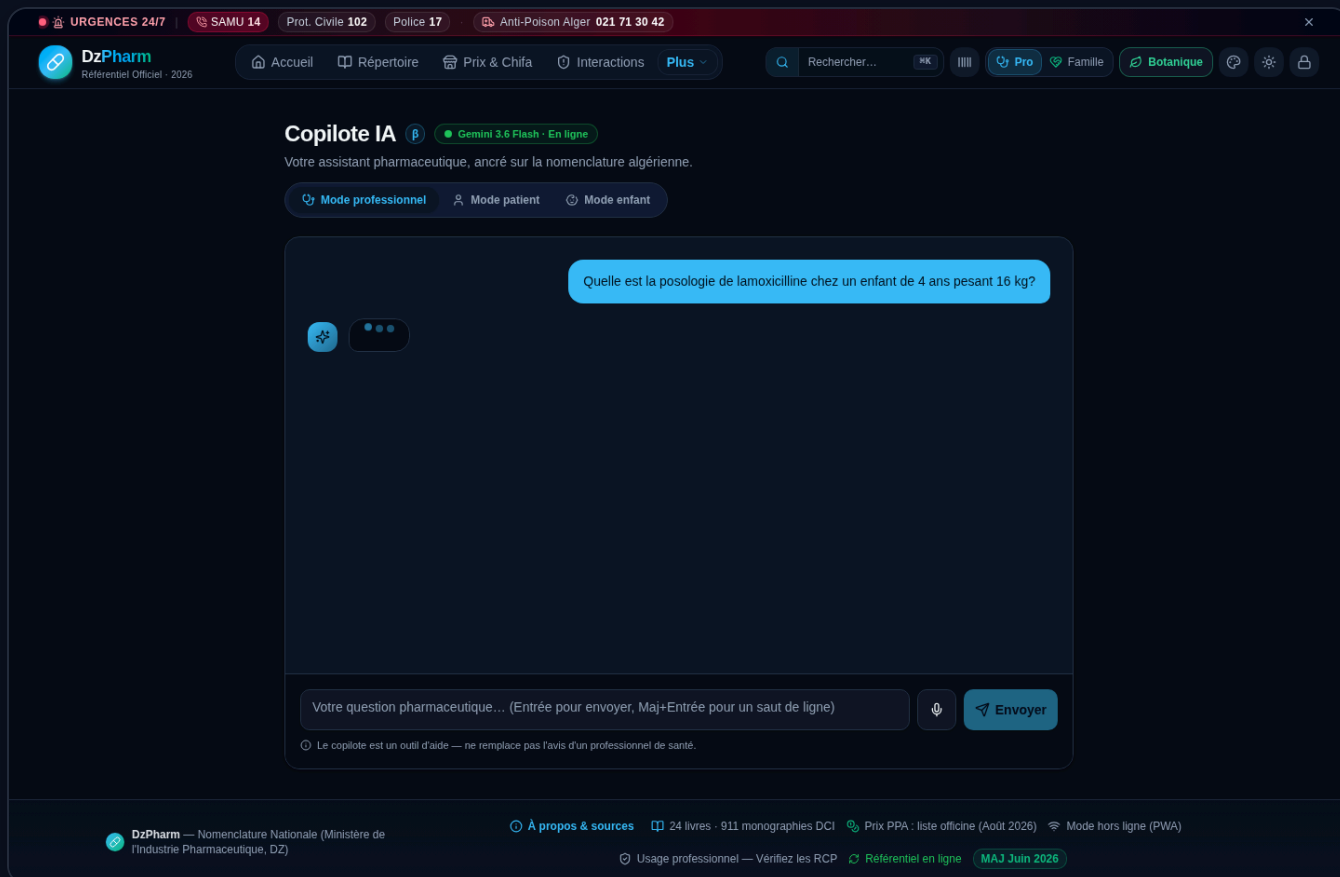


Figure 40.3 – A follow-up response. The Copilot correctly used context from the previous exchange to interpret the follow-up. But once the modal closes, this contextual continuity is lost – the next session starts fresh, with no memory of the previous exchange.

3.1 The Zustand Slice

```
// stores/copilotStore.ts - new slice for conversation history
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

interface Message {
  id: string;
  role: 'user' | 'assistant';
  content: string;
  mode: 'pro' | 'patient' | 'enfant';
  timestamp: number;
}

interface CopilotState {
  messages: Message[];
  addMessage: (msg: Message) => void;
  clearConversation: () => void;
  lastUpdated: number | null;
}

export const useCopilotStore = create<CopilotState>()(
  persist(
    (set) => ({
      messages: [],
      lastUpdated: null,
      addMessage: (msg) => set((s) => ({
        messages: [...s.messages, msg],
        lastUpdated: Date.now(),
      })),
      clearConversation: () => set({ messages: [], lastUpdated: null }),
    }),
    { name: 'dzpharm-copilot' }
  )
);
```

3.2 The Component Integration

On Copilot mount, read from the store. On every new message (user or assistant), call `addMessage`. Add a «Effacer la conversation» button that calls `clearConversation`. The wiring is straightforward — most of the work is in the store definition above.

3.3 Storage Limits & Pruning

LocalStorage has a ~5MB limit per origin. A conversation with 20 messages (10 user, 10 assistant) at ~2KB per message = ~40KB. Storing 100 past conversations = ~4MB — close to the limit. The store should prune: keep only the last 50 conversations, and within each, keep only the last 30 messages. This is a one-time configuration in the persist middleware. Effort: 1 hour to add the prune logic.

IMPLEMENTATION NOTE

The Zustand + persist pattern is already used for favorites (#48) and the interactions list (#22). Adding a `copilotHistory` slice is a 50-line addition to the existing store file. The harder part is the UI for the thread list (Pattern #2), which requires a sidebar component. Phase 1 (persistence only) can ship without the sidebar — the conversation simply survives refresh, which is already a major improvement.

CHAPTER 04 • CLINICAL RISK

The Patient-Continuity Argument

The missing history feature is not just a UX gap — it is a clinical-continuity risk. Three concrete scenarios illustrate why this matters more for a clinical Copilot than for a general chatbot.

4.1 Scenario A — Multi-Visit Patient Follow-Up

A pharmacist counsels a parent on pediatric paracetamol dosing for a 22 kg, 6-year-old child on Monday. The Copilot response is excellent — it gives the mg/kg calculation, the mL conversion, the Algerian pediatric forms, and the warnings. On Wednesday, the parent returns: the fever has not broken, the pharmacist wants to ask the Copilot about ibuprofen as a second-line option. Without history, the pharmacist must re-establish the full context (weight, age, current medications, the previous paracetamol discussion) from scratch. The re-establishment is error-prone. With history, the pharmacist reopens Monday's conversation and types «Et pour ibuprofène?» — the Copilot has the context.

4.2 Scenario B — Complex Drug Interaction Review

A pharmacist is reviewing a polypharmacy patient (5+ medications) and asks the Copilot about interactions. The Copilot provides a structured response. The pharmacist wants to follow up: «Quelle alternative pour le troisième médicament?» Without history, the follow-up requires restating the full medication list. With history, the follow-up is one sentence. For polypharmacy patients — common in elderly care — this is the difference between a 30-second follow-up and a 5-minute context rebuild.

4.3 Scenario C — Audit Trail

In a clinical setting, the question "what did the Copilot recommend?" may need to be answered weeks later — for example, if a patient returns with an adverse event and the pharmacist needs to recall what was discussed. Without history, there is no audit trail. With history + export (Pattern #4), the pharmacist can retrieve the past conversation and attach it to the patient record. This is not a hypothetical — it is a real workflow in any pharmacy that uses AI-assisted decision support.

CLINICAL RISK · NO AUDIT TRAIL

For a clinical Copilot, the absence of conversation history means there is no audit trail. A pharmacist who used the Copilot for a complex patient case cannot retrieve that discussion weeks later if an adverse event occurs. This is a real workflow gap, not a hypothetical. The fix (persistence + export) ships in 2 days and removes the audit-trail risk entirely.

Final Assessment & Phased Rollout

Conversation History is **the most consequential missing feature in the AI layer**. The Copilot is genuinely valuable for clinical questions (verified in #36, #37, #38, #39), but its statelessness makes it forget everything between sessions — which is unacceptable for a clinical tool. Total grade: **D+** — feature is essentially absent. Phase 1 (persistence, 1 day) lifts it to C+. Phase 2 (thread list + export, 4 days) lifts it to B+. Phase 3 (branching + search) lifts it to A-.

5.1 Phased Fix Plan

PHASE	FIXES	EFFORT	OUTCOME
Phase 1 · Now (Week 1)	Pattern #1 (persistence) — Zustand slice, survives refresh	~1 day	Grade lifts to C+. Conversation survives refresh. Clinical risk reduced.
Phase 2 · Next (Week 2-3)	Pattern #2 (thread list) + Pattern #4 (export)	~4 days	Grade lifts to B+. Sidebar of past conversations, export to Markdown/PDF.
Phase 3 · Later (Quarter 2)	Pattern #3 (branching) + Pattern #5 (search)	~2 weeks	Grade lifts to A-. Power-user patterns: forking, keyword search.

5.2 Cross-Feature Dependencies

- ✓ Feature #2 (State Management) — conversation history is a Zustand slice; #40 is a specific instance of the broader #2 pattern
- ✓ Feature #36 (AI Copilot Core) — #40 is the memory layer for #36's chat interface
- ✓ Feature #37 (Copilot Mode System) — mode should be stored per-message in the history
- ✓ Feature #48 (Favorites) — same Zustand+persist pattern; infrastructure already exists
- ✓ Feature #51-52 (Print/Export) — export pattern (#4) reuses the existing print pipeline
- ⚠ Feature #8 (Authentication) — server-side history requires per-user accounts (deferred until #8 ships)

5.3 Final Assessment

0/5

PATTERNS IMPLEMENTED

5

MISSING PATTERNS

D+

GRADE

~1d

PHASE 1 EFFORT

~4d

PHASE 2 EFFORT

~2w

PHASE 3 EFFORT

FINAL VERDICT · D+ GRADE, 1 DAY FROM C+

Conversation History is **the most consequential missing feature in the AI layer**. The Copilot's clinical value (#36, #37, #38, #39 deep-dives) is undermined by its statelessness — every refresh, every modal close, every tab close wipes the conversation. For a general chatbot this is an inconvenience; for a clinical Copilot it is a continuity risk and an audit-trail gap. The fix is mechanical (Zustand slice, 1 day) because the infrastructure already exists for favorites (#48). Phase 1 (persistence, 1 day) lifts the grade to C+. Phase 2 (thread list + export, 4 days) lifts it to B+. Phase 3 (branching + search, 2 weeks) lifts it to A-. The feature is worth investing in — clinical continuity is a real workflow need.

Immediate Next Steps

1. Add the `copilotHistory` Zustand slice with `persist` middleware — 1 day.
2. Wire the Copilot component to read/write from the slice on mount and on every message — 2 hours.
3. Add a «Effacer la conversation» button that calls `clearConversation` — 30 minutes.
4. Add prune logic (last 50 conversations, last 30 messages each) — 1 hour.
5. Defer thread list (Phase 2) and branching/search (Phase 3).

40

0 of 5 patterns implemented. Copilot is stateless — refresh wipes conversation. Clinical risk: no continuity, no audit trail. 1 day of work lifts to C+.

5

CHAPTERS

3

SCREENSHOTS

5

MISSING PATTERNS

D+

GRADE